CENG 201 – Task 2 Report

Topic: Treatment Queue Implementation using Queue Data Structure

Queue Behavior and Time Complexity Analysis:

In this task, a queue data structure was implemented to manage treatment requests in a hospital system. Each treatment request is added to the queue when a patient arrives and is processed in the same order. This behavior follows the First-In First-Out (FIFO) principle, which makes the queue a suitable data structure for handling treatment requests.

The enqueue operation adds a new treatment request to the end of the queue. Since the queue is implemented using a linked list and a reference to the rear node is maintained, this operation can be performed in O(1) time. Likewise, the dequeue operation removes the request from the front of the queue. As no traversal of the queue is required, its time complexity is also O(1).

The size method returns the number of treatment requests currently waiting in the queue. Because this value is stored and updated during enqueue and dequeue operations, the method runs in O(1) time. On the other hand, the printQueue method iterates through all elements in the queue to display them, resulting in a time complexity of O(n), where n is the number of requests in the queue.

Queue vs. Stack Comparison:

A queue is an appropriate data structure for managing treatment requests because patients should be treated in the order they arrive. This ensures fairness and prevents patients from waiting unnecessarily long. If a stack were used instead, the most recently added request would be processed first due to the Last-In First-Out (LIFO) principle. This would cause earlier patients to wait longer, which is not suitable for a hospital environment.

In terms of time complexity, both queue and stack operations such as insertion and removal can be performed in O(1) time. However, the queue provides the correct logical behavior for treatment scheduling, making it the more appropriate choice for this scenario.